Predictive Deadlock Warning: Early Detection of Unsafe States from Resource-Request Patterns

Revanth Reddy Lingala and Padigela Srujan
Department of Information Technology
Chaitanya Bharathi Institute of Technology (CBIT), Hyderabad, India

***Abstract*—Banker's algorithm tells an operating system whether the current resource-allocation state is *safe*; it gives no signal that the system is drifting towards an unsafe state. We ask how early, and how reliably, an unsafe state can be anticipated before the safety check reports it, and at what computational cost. We propose an explainable, rule-based early-warning score that combines five resource-pressure indicators with a persistence and refractory rule, and we compare it with a free-resource threshold, each indicator alone, a weighted-maximum variant, logistic regression and gradient-boosted trees trained on the same signals, a scalar safety margin in the spirit of Finkel and Madduri, a cheap top-3 lookahead, and an exact one-step lookahead that tests every running process. An observer-mode simulator grants every valid request so that unsafe states actually occur, and records Banker's safety and deadlock-detection outcomes after every step. Evaluation follows event-based early-warning practice: alarm precision, event recall, lead time, refractory periods, censoring, an NAB-style early-detection score and stratified bootstrap confidence intervals, with tuning and test on disjoint seeds. Over 195,000 simulated sequences (439.6 million requests) on AWS EC2, the rule-based score reached F1 = 0.28 [0.28, 0.28] with a median warning of 10 steps, against 0.16 for the free-resource threshold and 0.01 for random alarms at a matched rate; gradient-boosted trees trained on the *same* cheap signals reached 0.48 with a median warning of 11 steps, while logistic regression gained little (0.31), so non-linear interactions between the signals carry much of the missing information. State-based checks were more accurate still: on the sequences where all methods were computed, a scalar safety margin reached 0.72 against 0.74 for exact lookahead, at 1.7 versus 43.5 times the compute of a simulation step for 200 processes, but warned later. Results were insensitive to maximum claims misdeclared by ±20%, and unsafe episodes overwhelmingly ended in deadlock. Together the methods map a cost–accuracy–lead-time frontier for anticipating Banker's verdict.**

***Index Terms*—deadlock avoidance, Banker's algorithm, early warning, resource allocation, operating systems, event-based evaluation**

I. Introduction

Deadlock—processes holding resources while waiting for resources held by each other—remains a practical hazard wherever resources are allocated incrementally. Cluster schedulers that place the tasks of a gang-structured job one at a time can leave several jobs each holding part of what they need, a "temporary deadlock" reported in deep-learning platforms [1], [2] and the motivation for all-or-nothing placement in recent Kubernetes releases [3]. Resource exhaustion more broadly is a recurring cause of cloud outages [4].

The classical avoidance method, Banker's algorithm [5], [6], [7], checks whether a state is *safe*: whether some order exists in which every process can obtain its remaining maximum claim and finish. The check is binary and present-tense. It says *safe now* or *unsafe now*, but nothing about whether the system is drifting towards the boundary, and it is seldom used in practice because of its cost [8]. Operations teams, meanwhile, rely on early alerts that page before a resource saturates [9].

Prior work has made this check faster [8], [10], [11], characterised its computational complexity [12], [13], detected deadlocks after they form [14], [15], or predicted different kinds of deadlock, such as lock-order bugs in programs [16] or virtual-machine overload [17]. In all of it, the safety question remains present-tense. This paper asks a complementary, forward-looking question—analogous to a smoke detector that sounds before the fire alarm: *how early, how reliably and at what cost can an unsafe state be anticipated before Banker's algorithm reports it?* We study eight research questions:

- RQ1: Do indicators computed from the allocation state and request stream anticipate unsafe states?
- RQ2: How much lead time do alarms provide?
- RQ3: What alarm precision and event recall do they achieve?
- RQ4: Which indicators matter, how should they be combined, and do learned models on the same signals do better?
- RQ5: How does warning quality vary across workloads?
- RQ6: How often do unsafe states become deadlocks, and can alarms anticipate deadlocks?
- RQ7: What does each method cost, and how does the safety check scale?
- RQ8: How robust are the indicators when maximum claims are misdeclared?

Contributions. (i) An explainable, rule-based early-warning score for Banker's unsafe states, with no trained model. (ii) An observer-mode simulator with three controlled workloads, verified by 133 unit and property tests. (iii) An event-based evaluation protocol adapted from clinical, seismic and anomaly-detection early warning [18], [19], [20], [21]. (iv) A broad baseline set—matched-rate random alarms, a free-resource threshold, single indicators, learned models, a scalar safety margin, a cheap top-k lookahead and exact lookahead—that maps the *cost–accuracy frontier* for anticipating Banker's verdict. (v) A separate measurement of unsafe and deadlocked outcomes, a robustness study with misdeclared claims, and a scaling study, all run at scale on AWS EC2.

II. Background

A. Deadlock and Banker's algorithm

A system has m resource types with $total_j$ units each and n processes. Process i declares a maximum claim Max_{ij}, holds $Alloc_{ij}$ and may still request $Need_{ij} = Max_{ij} - Alloc_{ij}$; free units are $Avail_j = total_j - \sum_i Alloc_{ij}$. Deadlock requires mutual exclusion, hold-and-wait, no preemption and circular wait [22]; graph models and reduction-based detection are due to Holt [23]. In Banker's algorithm [6], [7], a state is safe if a sequence exists in which each process's need fits the free units plus everything released by the processes before it. The banker grants a request only if the resulting state is safe. The textbook safety check [24], [25] costs $\Theta(mn^2)$ [8].

B. Unsafe is not deadlock

The safety check is pessimistic: it assumes every process may request its full remaining claim. Deadlock detection is optimistic: it assumes running processes finish and release everything, so only blocked processes' pending requests matter [23], [24]. A state can be safe and not deadlocked, unsafe and not deadlocked, or unsafe and deadlocked, but never safe and deadlocked; safety is also not sufficient for liveness, since processes can still be postponed indefinitely [26], [27]. We therefore record both outcomes at every step and treat the unsafe onset as the primary prediction target and the deadlock onset as a secondary one.

III. Literature Survey and Positioning

We organise prior work into seven themes, then state precisely where our work sits (Table I).

A. Foundations: Banker's algorithm and the notion of safety

Deadlock handling is classically split into prevention, avoidance and detection [22], [23]. Prevention removes one of the four necessary conditions, for example by imposing a global order on resource acquisition [28]. Avoidance, our setting, originates with Dijkstra's Banker's algorithm [5], [6], generalised to multiple resource types by Habermann [7]. Holt, and Parnas and Habermann, showed that safety guarantees deadlock freedom but not freedom from indefinite postponement [26], [27]. *Relevance:* Dijkstra's definition of a safe state—a completion order exists in which every process obtains its remaining claim—is the exact ground truth our warnings try to anticipate. Every alarm, metric and baseline in this paper is defined relative to the moment this test first fails, so the original formulation is the reference point of the whole evaluation, not historical background.

B. Theory of deadlock prediction and avoidance

In the classical theory, "deadlock prediction" means deciding, from the current status of a resource allocation system, whether deadlock is still avoidable—the question Banker's algorithm answers. Gold showed that this decision is polynomial for reusable resources without partial requests (Banker's case) and NP-complete when either restriction is lifted [12]. Minoura developed avoidance algorithms that admit more concurrency than Havender's and Habermann's schemes and showed that avoidance for processes with branches is PSPACE-complete [13]. Kameda tested whether free-running processes can ever become deadlocked using network flow [29]. *Difference:* all of these works decide a *static, present-tense* property of a state—is it, or can it become, deadlocked or unsafe? We instead forecast *when* a running system will cross from safe to unsafe, from the trajectory of the allocation state, and measure how early and how reliably that crossing can be signalled.

C. Faster and dynamic safety tests

Because the safety check costs $\Theta(mn^2)$, most follow-up work makes it cheaper. Belik obtained an amortised $O(mn)$ variant under likely conditions and noted that avoidance is "seldom used on account of its high cost" [8]. Finkel and Madduri maintain safe sequences between allocator calls for an $O(\log n)$ test and compute the minimum resources a state needs to remain safe [10]. Li and Cook give near-linear detection algorithms when resource types are few [30], Lang extends the algorithm's applicability [31], and DBDAA admits processes dynamically and speeds up safe-state determination [11]. Andrews and Levin evaluate the safety of prospective requests between requests [32]. *Difference:* these works answer the same binary question faster; none anticipates the verdict. Two of them directly inspire our strongest baselines: our safety margin is a scalar, proportional form of Finkel and Madduri's minimum resources to remain safe, and our lookahead baselines follow Andrews and Levin's idea of testing prospective requests. We go beyond them by measuring what such checks buy as *early-warning signals* and what they cost.

D. Deadlock detection in databases and distributed systems

Practical systems often detect and recover instead of avoiding. In databases a deadlock is a cycle in the waits-for graph and can be detected cheaply [14], [33]; distributed detection uses probe messages along wait-for edges [15], [34]. *Difference:* detection reports a deadlock after it has formed; we warn before the system even becomes unsafe, and we report how often unsafe states actually turn into detected deadlocks.

E. Deadlock control beyond operating systems

Deadlock avoidance and prevention are mature in automated manufacturing, where Petri-net models of shared machines and buffers yield avoidance and prevention policies [35], [36], [37], [38]. Cluster managers allocate multi-dimensional resources to many concurrent jobs at datacentre scale [39], and the hold-and-wait pattern reappears there: deep-learning platforms report "temporary deadlocks" when gang jobs are placed task by task [1], [2], motivating all-or-nothing placement in Kubernetes [3], and load problems are a recurring cause of cloud outages [4]. *Relevance:* these works show the problem is not confined to textbooks and motivate our spike workload. *Difference:* they design control policies; we study prediction of the unsafe boundary and its cost.

F. Deadlock prediction in programs and learning-based prediction

In programming languages, "deadlock prediction" means finding lock-order deadlocks that some reordering of an observed execution could exhibit: lock-graph analysis [40], randomised active testing [41], sound prediction [42] and sound online prediction in linear time [16]. *Difference:* these tools find bugs in program code, involve single-unit locks with no declared claims, and answer whether a deadlock is *possible* in some interleaving; we monitor a running multi-unit resource allocator and forecast an imminent unsafe state. On the resource side, Le Ngoc and Tran Cong use linear regression in CloudSim to forecast virtual-machine "deadlock" for load balancing [17]; deadlock there means overload rather than Banker's unsafety, and neither lead time nor event-level precision is reported. AI-based thread-deadlock prediction has been proposed in preprints without empirical evaluation [43]. More broadly, failure prediction [44] and anomaly detection [45] learn to flag abnormal system states; our learned baselines use gradient boosting [46] on the same signals as our rule.

G. How early warnings are evaluated

A warning system must be judged on whether alarms come *before* events and how many are false—not on per-step accuracy, which a silent detector maximises when events are rare. Online failure prediction frames this through lead time and prediction windows [44]. *Relevance of clinical early warning:* Hyland et al. predict circulatory failure in intensive care with exactly our problem structure—a rare, harmful state change, a continuously monitored state, and value only if the alarm comes early [18]. They define event-level recall (events preceded by an alarm within a window), alarm-level precision (alarms followed by an event) and a silencing period after each alarm; we adopt these three definitions directly, and their reported operating point (about 30% precision) calibrates what "moderate" precision means for rare-event warning. *Relevance of seismic early warning:* Münchmeyer et al. evaluate earthquake alerts by alert quality *together with* warning time and show that lowering the alert threshold buys warning time at the cost of precision [19]; this is why we report precision against lead time along every threshold sweep (Fig. 4) and treat an alarm at the onset as a miss. Range-based precision and recall [20] and the Numenta Anomaly Benchmark score [21] motivate treating unsafe episodes as ranges and rewarding earlier detection. Because onsets are rare we use precision–recall rather than ROC analysis [47], [48], and bootstrap confidence intervals over whole sequences [49]. No deadlock study we found evaluates warnings this way, which is why these references come from medicine and seismology.

H. Positioning and novelty

Table I compares the closest prior work with ours. Prior work either decides safety or deadlock-freedom for the current state [12], [13], [29], decides it faster [8], [10], [11], detects deadlocks after they form [14], [15], predicts a different kind of deadlock (lock-order bugs in programs [42], [16] or VM overload [17]), or controls deadlock in other domains [36], [38]. To our knowledge, this is the first work to (i) forecast the safe-to-unsafe transition of a Banker-style resource allocator before the safety check fires, (ii) evaluate that forecast with event-based precision, recall and lead time against matched-rate random, threshold, learned and exact-lookahead baselines, and (iii) quantify the trade-off between accuracy, lead time and compute cost, including a cheap safety margin that nearly matches exact lookahead.

IV. Simulator

A. Observer mode

If Banker's algorithm refused unsafe requests, the system would never become unsafe and nothing could be predicted. Our simulator therefore grants every valid request and uses Banker's algorithm only to *observe*. Each step: (1) waiting processes are retried first-come first-served, and any process whose claim is now met finishes and releases everything, repeating until nothing changes; (2) one active, non-waiting process issues a non-zero request no larger than its need, which is granted if it fits the free units and otherwise blocks the process with the request pending; (3) completed processes release their resources; (4) in the *normal* workload a process may release part of its allocation early; (5) the simulator records whether the state is unsafe, whether it is deadlocked, the five indicators and the state-based baselines. A sequence ends at the first deadlock, when all processes finish, or at a step cap. Because no request is refused, observer mode also avoids a confound of clinical early warning, where interventions prevent events and turn correct alarms into apparent false positives [18].

B. Correctness

Initial states are regenerated until safe. Invariants—conservation of units, allocation $\leq$ claim, pending $\leq$ need, finished processes hold nothing—are asserted in a debug mode used by the tests. A vectorised NumPy safety check, which finishes every process that fits in each pass, agrees with the textbook loop on 2,000 random states, as does deadlock detection; both reproduce the textbook examples of [24]. The property *deadlocked* $\Rightarrow$ *unsafe* holds on every random state tested. Since detection assumes non-waiting processes release everything, grants to, releases by and completions of such processes cannot change its result; detection therefore reruns only when the waiting set or a pending request changes, which we verified against every-step detection. Recording the extra baselines draws on a separate random stream, so the simulated workload is identical with or without them. In total 133 tests cover every row of our edge-case matrix.

C. Workloads

Requests are fractions of remaining need, so behaviour is comparable across sizes. The main calibration parameter is claim tightness, $\sum_i Max_{ij}/total_j$. *Normal*: small single-resource requests and occasional partial releases. *Spike*: normal traffic, then—once a random 20–60% of the initial total need

TABLE I
POSITIONING AGAINST THE CLOSEST PRIOR WORK. "FORECAST" = PREDICTS A *FUTURE* TRANSITION RATHER THAN DECIDING A PROPERTY OF THE CURRENT STATE.

Work	Problem	Target	Forecast	Lead time	Event-based eval.	Cost analysis
Dijkstra; Habermann [6], [7]	Avoidance: is the state safe?	Banker's safety	–	–	–	–
Gold; Minoura [12], [13]	Complexity of deadlock prediction	avoidability	–	–	–	complexity
Belik; Finkel & Madduri; DBDAA [8], [10], [11]	Faster safety tests	Banker's safety	–	–	–	✓
Agrawal et al.; Chandy et al. [14], [15]	Deadlock detection	formed deadlock	–	–	–	✓
Kalhauge & Palsberg; Tunç et al. [42], [16]	Lock deadlocks in program traces	possible bug	–	–	–	✓
Le Ngoc & Tran Cong [17]	ML load balancing in CloudSim	VM overload	✓	–	–	–
Ezpeleta et al.; Li et al. [36], [38]	Petri-net deadlock control (manufacturing)	deadlock-free policy	–	–	–	–
Hyland et al.; Münchmeyer et al. [18], [19]	Clinical / seismic early warning	patient / earthquake event	✓	✓	✓	–
This work	**Early warning of Banker-unsafe states**	**unsafe (and deadlock) onset**	✓	✓	✓	✓

has been requested—80% of processes make broad grabs of 30–60% of their remaining need, capped by free units and holding back the last unit, a pattern akin to piecewise gang placement [1]. *Deadlock-prone*: process i prefers resource $(i{+}1) \bmod m$, inducing circular wait [22], and rarely takes its final unit. Parameters were calibrated on tuning seeds only so that each workload yields a mix of sequences with and without unsafe events (35%, 60% and 62% of test sequences contain at least one, averaged over sizes).

V. WARNING METHODS

A. Rule-based score

After every step five indicators in $[0, 1]$ are computed: *scarcity* $= 1 - \sum_j Avail_j / \sum_j total_j$; *need pressure* $= \frac{1}{m} \sum_j S_j/(S_j{+}Avail_j)$, with S_j the outstanding need of active processes; *contention*, the fraction of resource types for which at least two active processes need more than is free; *waiting ratio*, the fraction of active processes that are blocked; and *request rate*, the clipped relative increase in units requested over the last $W{=}10$ steps compared with the previous ten. The risk score is the weighted mean $100 \sum_i w_i x_i / \sum_i w_i$. An *alarm onset* occurs at step t if the score has been at least the threshold for k consecutive steps ending at t and no onset occurred in the previous R steps (refractory period, as in alarm silencing [18]). Alarms are suppressed while the system is already unsafe, since Banker's algorithm is then already signalling. The tuned score uses threshold 65, $k = 1$, $R = 10$ and weights "double_contention". Paging on simple, interpretable signals of near-saturation follows SRE practice [9].

B. Baselines

Every method maps per-step data to a 0–100 score and uses the same alarm rule.

- **Random:** independent alarms at each eligible step with a rate matched, on the tuning split, to the tuned combined score.
- **Availability:** the free fraction of units (identical to scarcity alone), the obvious threshold rule.
- **Single indicators** and a **weighted-maximum** combination $100 \max_i (w_i / \max w) x_i$, which lets the most alarming indicator decide.
- **Learned models:** logistic regression and gradient-boosted trees on the five indicators and their ten-step changes, labelled with "an unsafe onset follows within $H_{\max}$ steps". They are trained on even tuning seeds (497,560 class-balanced steps, prior-corrected to the true positive rate of 0.5%), their thresholds are tuned on odd tuning seeds, and they are reported on the test seeds.
- **Safety margin:** one minus the largest fraction of the free units that could be removed while the state stays safe, found by binary search (seven safety checks). It is a scalar, proportional version of Finkel and Madduri's minimum resources needed to remain safe [10].
- **Top-3 lookahead:** the fraction of the three running processes with the largest feasible grants for which granting that much would make the state unsafe (up to three safety checks).
- **Exact lookahead:** the same test for every running process (one safety check per process), in the spirit of evaluating prospective requests [32]. Because of its cost it is computed for all sequences with up to 25 processes and a seed subset for larger systems.

VI. EVALUATION METHOD

Events. An *unsafe onset* is a step where the state becomes unsafe; onsets after fewer than five safe steps are merged into the previous episode, as in range-based evaluation [20]. An alarm at step a is *true* if an onset falls in $[a{+}1, a{+}H_{\max}]$ with $H_{\max} = 20$; an onset is *captured* by the earliest unused alarm in its window, which gives its lead time [18]. An alarm at the onset step is a miss—detection, not prediction [19]. Alarms in the last $H_{\max}$ steps of a sequence cut by the step cap are *censored*; in sequences that ended in deadlock or completion

the future is known and such alarms count. Onsets in the first three steps are too early for any method and are excluded for all.

Metrics. We report micro-averaged alarm precision, event recall, F1, false alarms and alarms per 1,000 steps (alarm burden), lead-time distributions, and an NAB-style score [21] that rewards earlier detection: each captured onset earns a scaled sigmoid of its lead (close to 1 at the start of the window, near 0 just before onset), each false alarm costs 0.11 and each missed onset costs 1, normalised so that a silent detector scores 0 and perfect earliest detection scores 100. Confidence intervals are 95% percentile bootstraps over whole sequences, stratified by workload and size [49].

Tuning. Each method's threshold (5–95), $k \in \{1, 2, 3\}$ and $R \in \{10, 20\}$—and, for the combinations, the weight vector (equal, each indicator doubled, each removed)—are chosen on tuning seeds to maximise F1 subject to at most five false alarms per 1,000 steps and a median lead of at least three steps. All reported numbers come from a disjoint test-seed range.

Setup. Each workload ran at 5 sizes (10–200 processes, 5–20 resource types) with 10,000 test seeds and 3,000 tuning seeds per cell: 150,000 test and 45,000 tuning sequences, 439.6 million simulated requests in total (Table II). Experiments ran headless on one AWS EC2 c7i.8xlarge instance (32 vCPUs, 62 GB RAM, region ap-south-2) with Python, NumPy [50], scikit-learn and a process pool. The full pipeline—tests, calibration, simulation, model training, tuning, evaluation, scaling and figures—took 5 h 40 min and about $8.39 of compute. Runs record their configuration hash, code version (commit `328caf6`) and machine, and resume from chunked Parquet output after interruption.

VII. Results

A. RQ1–RQ3: early warning, lead time, precision and recall

Table III gives the main results. The rule-based score achieved F1 = 0.28 (precision 0.24, recall 0.34) with 0.33 false alarms per 1,000 steps, against 0.01 for random alarms at the same rate and 0.16 for the free-resource threshold: cheap indicators clearly carry predictive signal (RQ1). Captured onsets were warned a median of 10 steps ahead, 0.75 of them at least five steps ahead (RQ2). Precision and recall are moderate (RQ3), and Fig. 2 shows the full trade-off. For comparison, a deployed-quality clinical early-warning system operated at about 30% precision at its chosen recall [18].

B. RQ4: which signals, which combination, and learned models

Contention alone (F1 = 0.34) beat the weighted-mean score (0.28); need pressure (0.19), waiting ratio (0.25) and request rate (0.14) were weaker. The ablation (Table IV) shows that removing contention or request rate hurts the equal-weight mean most, while removing waiting ratio helps: averaging dilutes the strongest indicator, which is also why the tuner doubled contention's weight. The weighted maximum (0.34) selected weights under which contention always decides, and so coincides with contention alone. Learned models on the same signals tell a sharper story. Logistic regression reached only 0.31, but gradient-boosted trees reached 0.48 with an NAB score of 45.0 against 22.1 for the rule-based score, and on jointly captured onsets they warned earlier (median 12 vs. 9 steps). Since a linear model gains little and a tree ensemble gains much, the useful information lies in *interactions* between the indicators—for example contention matters most when waiting is also rising—which a single weighted sum cannot express. The price is explainability: the rule-based score can name the indicator that triggered an alarm, the tree ensemble cannot.

C. RQ7a: the cost–accuracy frontier

On the sequences where every method was computed (Table V), the state-based checks were the most accurate: exact lookahead reached F1 = 0.74, the safety margin 0.72 and top-3 lookahead 0.69, against 0.54 for boosted trees and 0.44 for the rule-based score. Their costs differ by more than an order of magnitude (Table VI, Fig. 3). The indicators and learned models need only a few vector operations per step. The safety margin costs 1.4× a simulation step on average and 1.7× at 200 processes, and top-3 lookahead 0.8× and 0.9×, because both use a fixed number of safety checks. Exact lookahead needs one check per running process, 6.4× on average and 43.5× at 200 processes. The safety margin therefore recovers almost all of exact lookahead's accuracy at about a twenty-fifth of its cost on large systems. Accuracy is not the only axis: on jointly captured onsets the state-based checks warned later than the rule-based score (median 3 vs. 9 steps for the margin), and Fig. 4 shows that every method trades precision for lead time along its threshold sweep.

D. RQ5: workloads

Warning quality depends strongly on how danger builds (Fig. 5, Table VII). Spike workloads were easiest for the rule-based score (F1 = 0.42): bursts raise several indicators at once, and unsafe onsets followed a median 26 steps after the burst began (only 3.0% occurred before it). Gradual build-ups were harder (0.22 for normal and 0.23 for deadlock-prone workloads); there, waiting ratio alone (0.29, 0.31) beat the weighted mean, and the state-based checks degraded least (safety margin 0.55 and 0.54), because they test the safety boundary directly instead of inferring it from aggregates.

E. RQ6: unsafe versus deadlock

In our model unsafe episodes rarely recover: 100% of resolved episodes ended in deadlock under spike and 100% under deadlock-prone workloads, and 77% under normal workloads, where early releases allow some recovery (Fig. 6). Processes eventually request their full claims, so Banker's worst-case assumption is realised. For the deadlock target, alarms are not suppressed during unsafe periods (Table VIII). With a 20-step horizon, waiting ratio and boosted trees anticipated deadlock best (F1 = 0.48 and 0.48), since blocked processes accumulate as a deadlock forms; with a 50-step horizon exact lookahead led (0.76 on its subset).

TABLE II

EXPERIMENTAL SETUP PER CELL (TEST SPLIT). TEST SPLIT: 150,000 SEQUENCES, 338,262,425 STEPS; TUNING SPLIT: 45,000 SEQUENCES, 101,341,468 STEPS. MACHINE: LINUX-7.0.0-1014-AWS-X86_64-WITH-GLIBC2.39 (32 CPUS, C7I.8XLARGE).

scenario	n	m	seqs	steps	events	% with event	% deadlock	tightness
normal	10	5	10,000	1,071,547	2,754	26.90	25.40	1.43
normal	25	8	10,000	4,494,155	1,674	15.70	14.30	1.43
normal	50	10	10,000	11,527,074	1,270	11.30	10.10	1.43
normal	100	20	10,000	39,280,274	8,469	65.80	62.70	1.43
normal	200	20	10,000	82,123,372	7,358	56.20	53.60	1.43
spike	10	5	10,000	707,662	7,066	70.70	70.70	1.58
spike	25	8	10,000	2,939,715	5,954	59.50	59.50	1.58
spike	50	10	10,000	7,434,425	5,317	53.20	53.20	1.58
spike	100	20	10,000	27,023,589	6,185	61.90	61.90	1.58
spike	200	20	10,000	56,973,051	5,346	53.50	53.50	1.58
deadlock_prone	10	5	10,000	1,020,230	4,073	40.70	40.70	1.48
deadlock_prone	25	8	10,000	4,036,769	4,047	40.50	40.50	1.48
deadlock_prone	50	10	10,000	9,877,410	4,513	45.10	45.10	1.48
deadlock_prone	100	20	10,000	29,793,804	9,316	93.20	93.20	1.48
deadlock_prone	200	20	10,000	59,959,348	9,270	92.70	92.70	1.48

TABLE III

MAIN RESULTS, UNSAFE TARGET, $H_{max} = 20$, TEST SPLIT, 95% BOOTSTRAP CIS.

method	precision	recall	F1 [95% CI]	FA/1000	lead med.	lead≥5	NAB	alarms
combined (ours)	0.24	0.34	0.28 [0.28, 0.28]	0.33	10	0.75	22.1	147,529
combined, max	0.30	0.39	0.34 [0.34, 0.34]	0.22	7	0.69	27.2	106,873
need pressure	0.31	0.14	0.19 [0.19, 0.19]	0.07	7	0.66	9.5	35,750
contention	0.30	0.39	0.34 [0.34, 0.34]	0.22	7	0.69	27.2	106,873
waiting ratio	0.24	0.26	0.25 [0.25, 0.25]	0.31	13	0.86	16.6	136,824
request rate	0.16	0.12	0.14 [0.13, 0.14]	0.15	9	0.75	6.6	59,822
availability	0.11	0.34	0.16 [0.16, 0.16]	0.71	8	0.72	13.0	268,034
logistic regr.	0.24	0.44	0.31 [0.31, 0.31]	0.48	12	0.86	28.3	213,974
boosted trees	0.41	0.58	0.48 [0.47, 0.48]	0.29	11	0.85	45.0	166,088
safety margin	0.64	0.61	0.62 [0.62, 0.63]	0.08	4	0.45	43.8	78,901
top-3 lookahead	0.54	0.62	0.58 [0.57, 0.58]	0.13	5	0.56	45.6	94,189
exact lookahead	0.69	0.81	0.74 [0.74, 0.75]	0.62	4	0.48	59.1	35,744
random	0.00	0.01	0.01 [0.01, 0.01]	0.44	11	0.82	–	148,994

TABLE IV

ABLATION: COMBINED SCORE WITH ONE INDICATOR REMOVED (EACH RE-TUNED ON THE TUNING SPLIT).

variant	precision	recall	F1	FA/1000
all five, equal	0.17	0.40	0.24	0.59
without scarcity	0.19	0.39	0.26	0.47
without need_pressure	0.17	0.45	0.25	0.64
without contention	0.14	0.36	0.20	0.65
without waiting_ratio	0.23	0.31	0.27	0.30
without request_rate	0.15	0.39	0.22	0.54

TABLE V

ALL METHODS ON THE SEQUENCES WHERE THE LOOKAHEAD BASELINE WAS COMPUTED.

method	precision	recall	F1	lead med.
combined (ours)	0.56	0.37	0.44 [0.44, 0.45]	5
combined, max	0.46	0.59	0.51 [0.51, 0.52]	6
need pressure	0.65	0.11	0.18 [0.18, 0.19]	4
contention	0.46	0.59	0.51 [0.51, 0.52]	6
waiting ratio	0.32	0.09	0.14 [0.14, 0.15]	4
request rate	0.44	0.27	0.33 [0.33, 0.34]	8
availability	0.48	0.22	0.30 [0.29, 0.30]	4
logistic regr.	0.23	0.79	0.36 [0.36, 0.37]	12
boosted trees	0.40	0.81	0.54 [0.53, 0.54]	10
safety margin	0.68	0.77	0.72 [0.71, 0.72]	4
top-3 lookahead	0.63	0.77	0.69 [0.69, 0.70]	5
exact lookahead	0.69	0.81	0.74 [0.74, 0.75]	4
random	0.03	0.01	0.01 [0.01, 0.01]	9

F. RQ8: misdeclared claims

Banker's algorithm needs declared maximum claims, which real systems rarely know. We fed the tuned rules indicators computed from claims misdeclared by up to $\pm 20\%$ per entry while the ground truth used the true claims (Table IX, Fig. 7). F1 changed by at most half a percentage point for every method (0.000 for the rule-based score, -0.001 for contention, -0.001 for boosted trees). The indicators aggregate need over all processes and resource types, so per-entry claim errors largely cancel; this partly answers the objection that Banker-style signals require exactly known claims.

G. Sensitivity to the warning horizon

With the same alarms, widening the horizon raises F1 for every method (rule-based score: 0.11, 0.19, 0.28 and 0.40 for H_{max} = 5, 10, 20, 50; Fig. 8), showing that many alarms precede onsets by more than 20 steps.

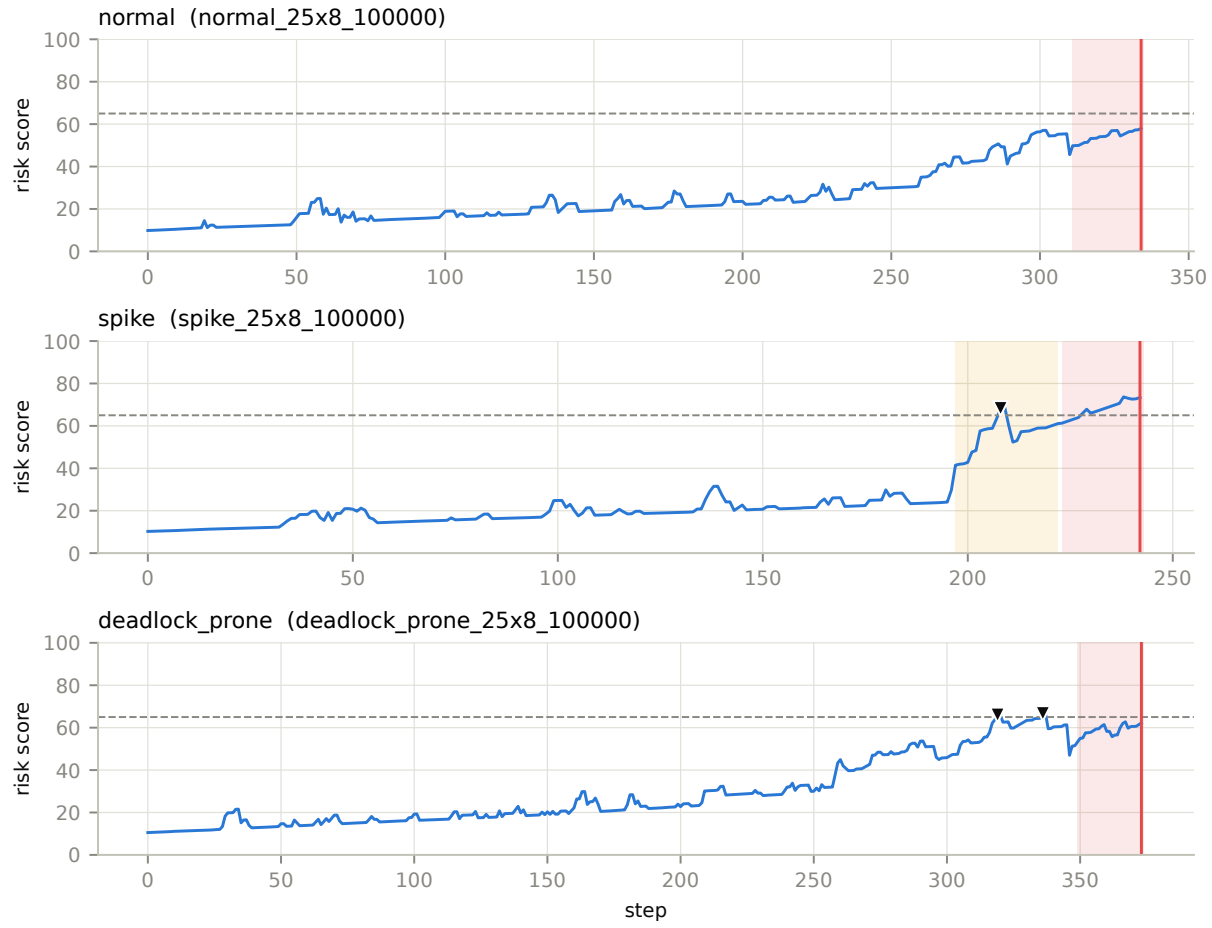


Fig. 1. Example test sequences (25 processes, 8 resource types), one per workload. Blue: rule-based risk score; dashed: tuned threshold; triangles: alarm onsets; red band: unsafe episode; red line: deadlock; amber band: spike burst.

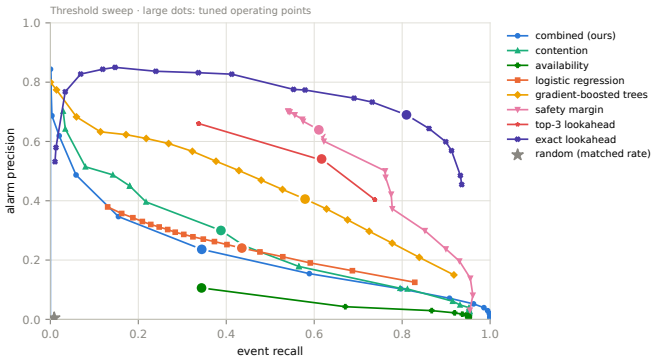


Fig. 2. Precision–recall along each method's threshold sweep (plotted on test data, never used for selection; points are not interpolated [48]). Large dots: tuned operating points; star: random baseline.

TABLE VI
MEASURED COMPUTE PER STEP (TEST SPLIT). A SIMULATION STEP INCLUDES ONE SAFETY CHECK AND THE INDICATORS; LEARNED MODELS ADD A FEW VECTOR OPERATIONS.

computation	processes	μs/step	× simulation step
simulation step	all	289.18	1.00
one safety check	all	59.89	0.21
safety margin	all	417.28	1.44
safety margin	200	479.30	1.66
top-3 lookahead	all	231.46	0.80
top-3 lookahead	200	264.00	0.91
exact lookahead	all	1850.09	6.40
exact lookahead	200	12565.63	43.45

H. RQ7b: scaling of the safety check

The vectorised safety check was 1.4× (10 processes) to 7.7× (200 processes) faster than the textbook loop (Table X, Fig. 9). Measured growth with n was far below the $\Theta(mn^2)$ worst case (log–log slopes 0.27 vectorised and 0.76 reference at $m = 20$): realistic states clear in a few passes, consistent with Belik's observation that typical cases are cheaper than the worst case [8].

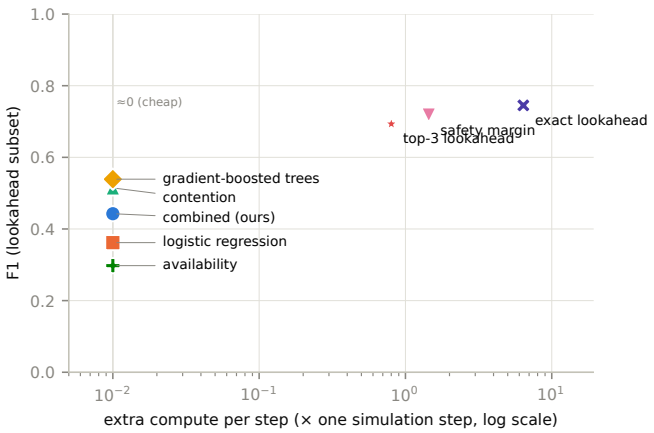


Fig. 3. Cost–accuracy frontier: F1 on the sequences where all methods were computed, against extra compute per step (relative to one simulation step, log scale). Cheap signals are drawn at the axis floor.

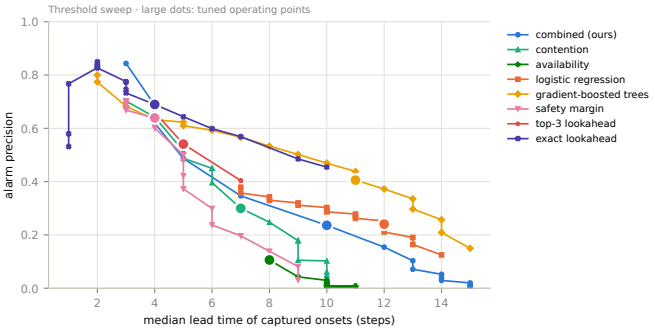


Fig. 4. Alarm precision against median lead time along each method's threshold sweep [19]. Large dots: tuned operating points.

VIII. Discussion

Circularity. The indicators, the learned models and Banker's check all read the same allocation state, so some correlation is built in. That is the point of the question: can cheap aggregates of that state anticipate an expensive verdict before it flips? The baselines bound the answer from both sides—random alarms and the free-resource rule from below, exact lookahead from above—and the state-based checks show how much of the information in the state cheap aggregates leave unused.

A frontier, not a winner. No method dominates on accuracy, cost, lead time and explainability at once. The rule-based score is cheap and fully explainable but only moderately precise; boosted trees on the same cheap signals are markedly better and warn early but are opaque; the safety margin is nearly as accurate as exact lookahead at a fraction of its cost but warns later. A practical design follows directly: run a cheap score continuously and invoke the safety margin, or exact lookahead on small systems, only when the score is elevated, paying the safety-check cost only in risky periods.

Explainability. A rule-based alarm can be traced to the indicator that triggered it, which matters for alerts that page humans [9].

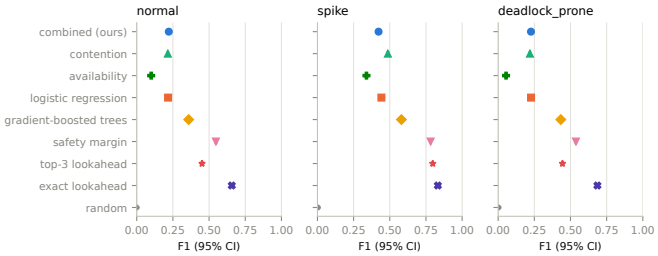


Fig. 5. F1 per workload with 95% bootstrap confidence intervals.

TABLE VII
F1 per scenario (unsafe target, $H_{max} = 20$).

method	normal	spike	deadlock_prone
combined (ours)	0.22	0.42	0.23
combined, max	0.21	0.49	0.22
need pressure	0.06	0.34	0.01
contention	0.21	0.49	0.22
waiting ratio	0.29	0.03	0.31
request rate	0.00	0.23	0.00
availability	0.10	0.34	0.05
logistic regr.	0.22	0.44	0.23
boosted trees	0.36	0.58	0.43
safety margin	0.55	0.78	0.54
top-3 lookahead	0.45	0.80	0.44
exact lookahead	0.66	0.83	0.69
random	0.00	0.01	0.01

IX. Threats to Validity

Synthetic workloads. Workloads are synthetic; we mitigate this with three controlled patterns, one modelled on reported gang-scheduling behaviour [1], and per-workload reporting. **Known claims.** Banker's algorithm needs declared maximum claims [6], an inherited limitation also noted for DBDAA [11]; we quantify sensitivity to misdeclared claims. **Calibration.** Claim tightness, which largely decides whether a workload becomes unsafe, was calibrated for class balance on tuning seeds. **Modelling choices.** Flicker merging, the warm-up exclusion, the refractory period and ending a sequence at the first (possibly partial) deadlock are design decisions taken from early-warning practice [18], [20]; the horizon study shows their direction of effect. **Subset comparison.** Exact lookahead was computed on a seed subset for large systems; we therefore compare all methods on that same subset. **Dependence.** Steps within a sequence are correlated, so the bootstrap resamples whole sequences.

X. Conclusion and Future Work

Simple, explainable indicators anticipate Banker-unsafe states several steps ahead, clearly better than chance and than a free-resource rule, at negligible cost and robustly to misdeclared claims. A non-linear model on the same signals improves accuracy substantially, showing that indicator interactions matter, and a scalar safety margin nearly matches exact lookahead at a small, size-independent number of safety

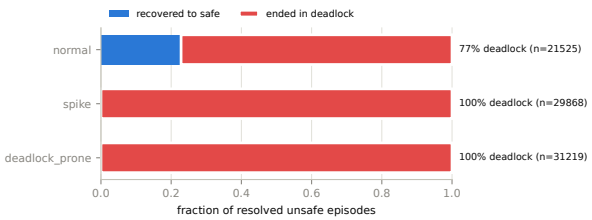


Fig. 6. Outcome of resolved unsafe episodes per workload.

TABLE VIII
DEADLOCK-ONSET TARGET ($H_{max} = 20$, ALARMS NOT MASKED WHILE UNSAFE).

method	precision	recall	F1	lead med.	events
combined (ours)	0.21	0.71	0.33	13	77,700
combined, max	0.17	0.54	0.26	12	77,700
need pressure	0.18	0.34	0.23	10	77,700
contention	0.17	0.54	0.26	12	77,700
waiting ratio	0.33	0.83	0.48	12	77,700
request rate	0.07	0.06	0.06	15	77,700
availability	0.10	0.52	0.16	10	77,700
logistic regr.	0.18	0.65	0.29	14	77,700
boosted trees	0.31	0.99	0.48	14	77,700
safety margin	0.14	0.17	0.16	14	77,700
top-3 lookahead	0.11	0.15	0.13	15	77,700
exact lookahead	0.32	0.43	0.37	15	25,862
random	0.00	0.01	0.01	10	77,700

checks. Together these results map the cost–accuracy–lead-time frontier for anticipating Banker's verdict. Future work includes two-stage warning (cheap score gating a lookahead), evaluation on real allocation traces, estimating claims rather than assuming them, incremental safety checks [8], [10], [11], and an avoidance variant that acts on warnings.

REFERENCES

[1] K. R. Jayaram, V. Muthusamy, P. Dube, V. Ishakian, C. Wang, B. Herta, S. Boag, D. Arroyo, A. Tantawi, A. Verma, F. Pollok, and R. Khalaf, "FfDL: A flexible multi-tenant deep learning platform," in *Proc. 20th Int. Middleware Conference (Middleware '19)*, 2019, pp. 82–95.

[2] C.-Y. Lin, T.-A. Yeh, and J. Chou, "DRAGON: A dynamic scheduling and scaling controller for managing distributed deep learning jobs in Kubernetes cluster," in *Proc. 9th Int. Conf. on Cloud Computing and Services Science (CLOSER)*. SciTePress, 2019, pp. 569–577.

[3] Kubernetes Project, "Kubernetes v1.35: Introducing workload aware scheduling," https://kubernetes.io/blog/2025/12/29/kubernetes-v1-35-introducing-workload-aware-scheduling/, 2025.

[4] H. S. Gunawi, M. Hao, R. O. Suminto, A. Laksono, A. D. Satria, J. Adityatama, and K. J. Eliazar, "Why does the cloud stop computing? lessons from hundreds of service outages," in *Proc. 7th ACM Symposium on Cloud Computing (SoCC)*, 2016, pp. 1–16.

[5] E. W. Dijkstra, "Cooperating sequential processes," Technological University, Eindhoven, Tech. Rep. EWD-123, 1965.

[6] ——, "The mathematics behind the Banker's algorithm (EWD 623)," https://www.cs.utexas.edu/~EWD/transcriptions/EWD06xx/EWD623.html, 1977, in *Selected Writings on Computing: A Personal Perspective*, Springer, 1982, pp. 308–312.

[7] A. N. Habermann, "Prevention of system deadlocks," *Communications of the ACM*, vol. 12, no. 7, pp. 373–377, 385, 1969.

[8] F. Belik, "Deadlock avoidance with a modified banker's algorithm," *BIT Numerical Mathematics*, vol. 27, no. 3, pp. 290–305, 1987.

[9] R. Ewaschuk, "Monitoring distributed systems," in *Site Reliability Engineering: How Google Runs Production Systems*, B. Beyer, C. Jones, J. Petoff, and N. R. Murphy, Eds. O'Reilly, 2016.

TABLE IX
ROBUSTNESS: THE SAME TUNED RULES WHEN NEED PRESSURE AND CONTENTION ARE COMPUTED FROM MAXIMUM CLAIMS MISDECLARED BY UP TO $\pm 20\%$ (GROUND TRUTH USES THE TRUE CLAIMS).

method	F1, true claims	F1, claims $\pm 20\%$	change
combined (ours)	0.280	0.280	+0.000
combined, max	0.338	0.337	-0.001
need pressure	0.190	0.194	+0.004
contention	0.338	0.337	-0.001
logistic regr.	0.310	0.311	+0.002
boosted trees	0.477	0.476	-0.001

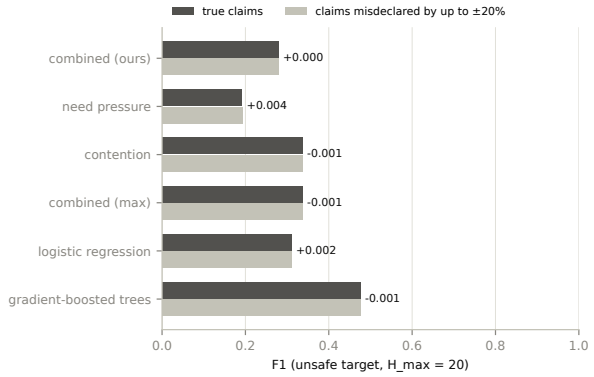


Fig. 7. F1 with true and with misdeclared ($\pm 20\%$) maximum claims, using the same tuned rules.

TABLE X
RUNTIME SCALING: TIME PER BANKER'S SAFETY CHECK ON RANDOM VALID STATES (VECTORISED VS. REFERENCE) AND WHOLE-SIMULATOR TIME PER STEP.

n	m	vec μs	ref μs	speed-up	μs/step
10	5	7.1	9.7	1.4	100.7
10	10	6.9	9.4	1.4	103.0
10	20	6.9	10.5	1.5	103.7
25	5	9.0	19.5	2.2	101.2
25	10	7.4	17.6	2.4	104.6
25	20	7.4	17.3	2.3	107.5
50	5	9.1	29.6	3.3	102.1
50	10	8.1	28.4	3.5	113.3
50	20	9.0	28.6	3.2	118.8
100	5	12.1	60.8	5.0	110.0
100	10	9.9	51.9	5.3	118.0
100	20	11.2	54.3	4.9	118.9
200	5	17.8	138.0	7.7	129.1
200	10	13.7	103.3	7.6	131.7
200	20	15.6	99.7	6.4	141.6

[10] R. Finkel and H. H. Madduri, "An efficient deadlock avoidance algorithm," *Information Processing Letters*, vol. 24, no. 1, pp. 25–30, 1987.

[11] M. F. Zohora, F. Farhin, and M. S. Kaiser, "DBDAA: A real-time approach to dynamic Banker's deadlock avoidance algorithm with optimized time complexity," *PLOS ONE*, vol. 19, no. 9, p. e0310807, 2024.

[12] E. M. Gold, "Deadlock prediction: Easy and difficult cases," *SIAM Journal on Computing*, vol. 7, no. 3, pp. 320–336, 1978.

[13] T. Minoura, "Deadlock avoidance revisited," *Journal of the ACM*, vol. 29, no. 4, pp. 1023–1048, 1982.

[14] R. Agrawal, M. J. Carey, and D. J. DeWitt, "Deadlock detection is cheap," *ACM SIGMOD Record*, vol. 13, no. 2, pp. 19–34, 1983.

[15] K. M. Chandy, J. Misra, and L. M. Haas, "Distributed deadlock detection," *ACM Transactions on Computer Systems*, vol. 1, no. 2, pp. 144–156, 1983.

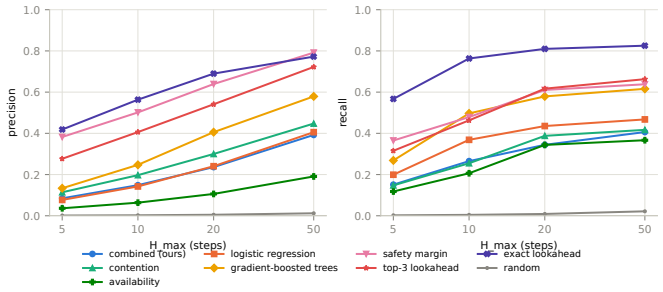


Fig. 8. Precision and recall as the warning horizon $H_{\max}$ varies (same alarms).

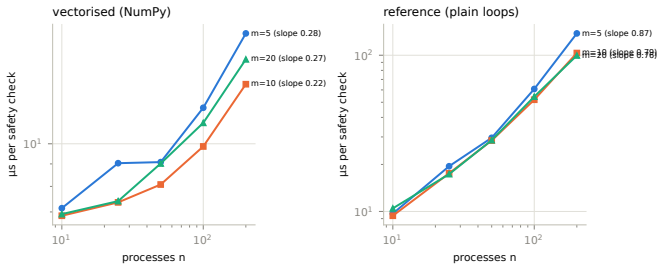


Fig. 9. Time per safety check versus number of processes (log–log), vectorised and reference implementations.

[16] H. C. Tunç, U. Mathur, A. Pavlogiannis, and M. Viswanathan, "Sound dynamic deadlock prediction in linear time," *Proceedings of the ACM on Programming Languages*, vol. 7, no. PLDI, pp. 1733–1758, 2023.

[17] H. L. Ngoc and H. T. Cong, "Enhancing load balancing in cloud computing through deadlock prediction," in *Industrial Networks and Intelligent Systems (INISCOM 2023)*, ser. LNICST, vol. 531. Springer, 2023, pp. 257–274.

[18] S. L. Hyland, M. Faltys, M. Hüser, X. Lyu, T. Gumbsch, C. Esteban, C. Bock, M. Horn, M. Moor, B. Rieck, M. Zimmermann, D. Bodenham, K. Borgwardt, G. Rätsch, and T. M. Merz, "Early prediction of circulatory failure in the intensive care unit using machine learning," *Nature Medicine*, vol. 26, no. 3, pp. 364–373, 2020.

[19] J. Münchmeyer, D. Bindi, U. Leser, and F. Tilmann, "The transformer earthquake alerting model: A new versatile approach to earthquake early warning," *Geophysical Journal International*, vol. 225, no. 1, 2021.

[20] N. Tatbul, T. J. Lee, S. Zdonik, M. Alam, and J. Gottschlich, "Precision and recall for time series," in *Advances in Neural Information Processing Systems 31 (NeurIPS)*, 2018, pp. 1924–1934.

[21] A. Lavin and S. Ahmad, "Evaluating real-time anomaly detection algorithms – the Numenta anomaly benchmark," in *Proc. IEEE 14th Int. Conf. on Machine Learning and Applications (ICMLA)*, 2015, pp. 38–44.

[22] E. G. Coffman, M. Elphick, and A. Shoshani, "System deadlocks," *ACM Computing Surveys*, vol. 3, no. 2, pp. 67–78, 1971.

[23] R. C. Holt, "Some deadlock properties of computer systems," *ACM Computing Surveys*, vol. 4, no. 3, pp. 179–196, 1972.

[24] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Wiley, 2018.

[25] A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, 5th ed. Pearson, 2022.

[26] R. C. Holt, "Comments on prevention of system deadlocks," *Communications of the ACM*, vol. 14, no. 1, pp. 36–38, 1971.

[27] D. L. Parnas and A. N. Habermann, "Comment on deadlock preventive method," *Communications of the ACM*, vol. 15, no. 9, pp. 840–841, 1972.

[28] J. W. Havender, "Avoiding deadlock in multitasking systems," *IBM Systems Journal*, vol. 7, no. 2, pp. 74–84, 1968.

[29] T. Kameda, "Testing deadlock-freedom of computer systems," *Journal of the ACM*, vol. 27, no. 2, pp. 270–280, 1980.

[30] Y. Li and R. Cook, "A new algorithm and asymptotical properties for the deadlock detection problem for computer systems with reusable resource types," in *Advances and Innovations in Systems, Computing Sciences and Software Engineering*. Springer, 2007, pp. 509–512.

[31] S.-D. Lang, "An extended banker's algorithm for deadlock avoidance," *IEEE Transactions on Software Engineering*, vol. 25, no. 3, 1999.

[32] G. R. Andrews and G. M. Levin, "On-the-fly deadlock prevention," in *Proc. 1st ACM Symposium on Principles of Distributed Computing (PODC)*, 1982.

[33] E. Knapp, "Deadlock detection in distributed databases," *ACM Computing Surveys*, vol. 19, no. 4, pp. 303–328, 1987.

[34] M. Singhal, "Deadlock detection in distributed systems," *Computer*, vol. 22, no. 11, pp. 37–48, 1989.

[35] Z. A. Banaszak and B. H. Krogh, "Deadlock avoidance in flexible manufacturing systems with concurrently competing process flows," *IEEE Transactions on Robotics and Automation*, vol. 6, no. 6, pp. 724–734, 1990.

[36] J. Ezpeleta, J. M. Colom, and J. Martínez, "A Petri net based deadlock prevention policy for flexible manufacturing systems," *IEEE Transactions on Robotics and Automation*, vol. 11, no. 2, pp. 173–184, 1995.

[37] M. P. Fanti and M. Zhou, "Deadlock control methods in automated manufacturing systems," *IEEE Transactions on Systems, Man, and Cybernetics, Part A*, vol. 34, no. 1, pp. 5–22, 2004.

[38] Z. W. Li, M. C. Zhou, and N. Q. Wu, "A survey and comparison of Petri net-based deadlock prevention policies for flexible manufacturing systems," *IEEE Transactions on Systems, Man, and Cybernetics, Part C*, vol. 38, no. 2, pp. 173–188, 2008.

[39] A. Verma, L. Pedrosa, M. Korupolu, D. Oppenheimer, E. Tune, and J. Wilkes, "Large-scale cluster management at Google with Borg," in *Proc. 10th European Conference on Computer Systems (EuroSys)*, 2015, pp. 1–17.

[40] S. Bensalem and K. Havelund, "Dynamic deadlock analysis of multi-threaded programs," in *Hardware and Software Verification and Testing (HVC 2005)*, ser. LNCS. Springer, 2006, pp. 208–223.

[41] P. Joshi, C.-S. Park, K. Sen, and M. Naik, "A randomized dynamic program analysis technique for detecting real deadlocks," in *Proc. 30th ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*, 2009, pp. 110–120.

[42] C. G. Kalhauge and J. Palsberg, "Sound deadlock prediction," *Proceedings of the ACM on Programming Languages*, vol. 2, no. OOPSLA, pp. 1–29, 2018.

[43] J. Oluwaseyi, "AI-enhanced predictive systems for thread deadlock resolution: Early detection and prevention in cloud applications," EasyChair Preprint 14955 (not peer-reviewed), 2024. [Online]. Available: https://easychair.org/publications/preprint/fQbd/open

[44] F. Salfner, M. Lenk, and M. Malek, "A survey of online failure prediction methods," *ACM Computing Surveys*, vol. 42, no. 3, pp. 10:1–10:42, 2010.

[45] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM Computing Surveys*, vol. 41, no. 3, pp. 15:1–15:58, 2009.

[46] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *The Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.

[47] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," *PLoS ONE*, vol. 10, no. 3, p. e0118432, 2015.

[48] J. Davis and M. Goadrich, "The relationship between precision-recall and ROC curves," in *Proc. 23rd Int. Conf. on Machine Learning (ICML)*, 2006, pp. 233–240.

[49] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. Chapman & Hall/CRC, 1993.

[50] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers *et al.*, "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, 2020.